Improving SOA services availability by Multi Agent Systems (MAS)

Roger Tang
Instituto de Pesquisas Tecnológicas do Estado de São Paulo(IPT)
Av. Prof. Almeida Prado, 532 – 055.08-901 – São Paulo – SP – Brasil
rogertangrt@hotmail.com

Marcelo Novaes de Rezende
Instituto de Pesquisas Tecnológicas do Estado de São Paulo(IPT)
Av. Prof. Almeida Prado, 532 – 055.08-901 – São Paulo – SP – Brasil
mrezende@linkededucation.com.br

Services Oriented Architecture (SOA) is an architectural software pattern. Sharing services for business process automation is an important SOA feature. These shared services may fail and, in this case, the business process involved may be affected. So, it's important that the services can offer high availability and rather be fail safe. This work presents a proposal based on Multi Agent System (MAS) to improve the SOA services availability. The proposal introduces an intelligence layer in the SOA reference architecture (OPEN GROUP, 2011). This intelligence layer, using self-healing techniques based on autonomic computing, allows recovering the system from failure situations. A prototype was built in order to exemplify the proposal and for partial requirements validation.

Keywords— Service Oriented Architecture, Multi-Agent system, Autonomic Computing, Availability

I. Introduction

SOA is a software development methodology that includes guidelines, standards and best practices for building services. According to Papazoglou [1], SOA is founded on the modeling of information systems based on business processes. According the model proposed by OPEN GROUP [2], SOA is inserted in a reference architecture that serves to define its essence and propose a common understanding vocabulary. This architecture provides a normative reference that remains relevant to SOA as an abstract and powerful model that is independent of the various and inevitable technological evolutions that will influence its realization.

Each service has an important role in the business process, so one of the problems of distributed computing is when the unavailability of a single service results in the unavailability of a business process or even the system as a whole. There at, the availability of SOA services become a challenge and should receive a lot of attention in a corporate environment. The web services environments require a greater availability of the applications beyond introducing a greater complexity in terms of access and management of the services [3]. Problems arise when there are critical processes that requiring high availability needs guarantee of consistent answers. Therefore it is expected that the services will be reliable, and if any failure happens there is an automatic behavior to recovery the error and reestablish the system to its normal operation's state, since large waits time can lead to financial losses.

Franke [4] presents the economic importance of availability management and the implications for SLA management under the aspect of changing service interruption costs. He concludes based on revenue data that when outage costs are proportional to outage duration, more but shorter outages should be preferred to fewer but longer, in order to minimize variance.

II. Autonomic Computing

The Autonomic Computing (AC) paradigm has been inspired by the human autonomic nervous system that controls and monitors changes inside and outside our body, integrating the stimuli to the reactions. The central objective of this behavior is related to the survival of the system. If a stimulus leads the system to a state of imbalance, the autonomous system works towards the resumption of this equilibrium [5].

According White [6], the main properties of autonomous computing are:

- Self-management;
- Self-configuration;
- Self-healing;
- Self-optimization;
- Self-protection.

An autonomous computing system requires sensors to perceive environment's changes and need to have an intelligence to react to such changes in order to bring back or equilibrium to the system. The stimulus perceived by the sensors must be analyzed in order to determine if there is a malfunction, and if so, a plane must be triggered to determine which behavioral changes must be taken to take back the stability. Self-healing in particular helps to increase the service's availability and reliability, because it provides a mechanism to detect, diagnose and correct faults automatically, keeping the system consistent. Because that, it is desirable for an SOA to have the attributes of an autonomous architecture.

A. MAPE-K Model

The (Monitor, Analyze, Plan, Execute, and Knowledge - MAPE-K) model is an autonomic computer reference and it is used to management of autonomous elements. This model was initially proposed by IBM through the article "An architectural blueprint for autonomic computing" [5]. This model defines an autonomic manager that manages resources. These resources can be any part of the system. The two main elements are the Managed element and the Autonomic manager [6].

In this model, a standard execution cycle is run. Initially, the Monitor collects information about managed element; afterwards, the Analyzer identifies the current situation and generates a correction plan if any fail occurs and sequentially

the planner specifies which actions will be performed for recovery the system. Finally, the Executor recommends that actions need to be performed for the managed element. The four phases share information about the managed element [7].

Autonomous computing management involves a control module that has the following steps:

- Monitor or Data Extract;
- Data Analyze
- Decision Plan
- Decision Execution
- Knowledge Base.

III. SOA Intelligence Layer

The SOA reference architecture proposed by the Open Group Fig. 1 has nine layers of the reference architecture, with describing the main responsibilities of the layers and their primary building blocks which typically emerge from a process of creating or defining an architectural pattern [2]. Each layer is designed to facilitate the achievement of each of the prospects of a SOA architecture are:

- Cost reduction;
- Agility;
- Increased competitive advantage;
- Time-to-Market;
- Consolidation;
- Alignment.

The intention is to extend the SOA reference architecture model to increase the availability of web services. Because the quality of service layer is aims to capture and monitor operational metrics, ensuring availability, control, stability, security and reliability, this layer was selected to be the basis of the extension. Therefore, the intelligence layer is a proposal to extension of the quality of service layer that uses the MAPE-K reference model proposed by Kephart and Chess [6] that uses multi-agent system (MAS) techniques.

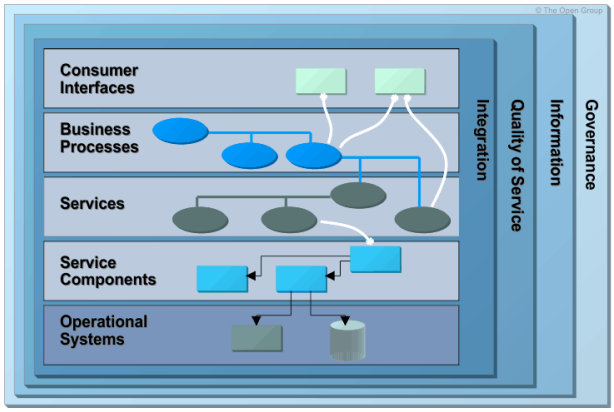

Fig. 1 Logical view of the SOA reference architecture solution [2]

As can be seen in Fig. 2 the intelligence layer is contained within the quality of service layer. As the reference architecture proposed by the Open Group is defined in partial layers, it is not necessary to access all the elements of a layer. This fact makes it possible to use the intelligence layer by any service of the reference model.

When a failure occurs, the intelligence layer receives requests from the interface and it acts to correct the failed service. It is also important to point out that there must be a service that plays the role of client (consumer of a resource), and a service that plays the role of server (service provider). The resource that providing the service and which must be protected by the agents will be called Monitored Resource in the intelligence layer in reference of the Managed Element in the MAPE-K model.

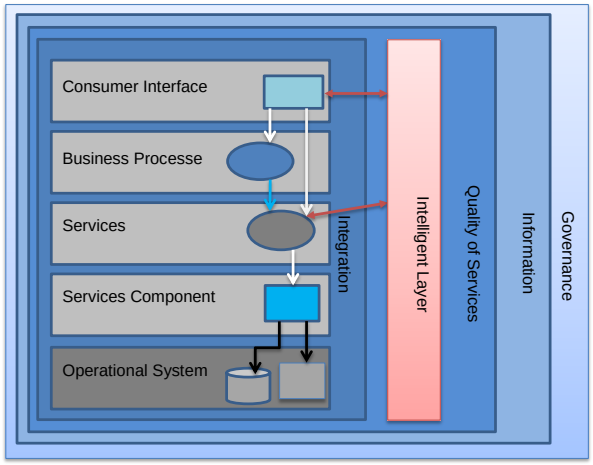

Fig. 2 Intelligent Layer (Produced by the author)

A. *Intelligenc Layer Requirements*

The intelligence layer has some important functional requirements to be built, most of them came from MAPE-K. These requirements are presented following:

(**RF1**) Failure Detection: It is necessary that the system detect to manifest a failure when it occurs. (Monitoring - MAPE-K).

(RF2) Transaction: It is necessary that the system has a way to return to a consistent state if it's impossible to recover a failure

(**RF3**) Data Analyze: It is necessary that the system has a way of analyzing the collected information. (Data analysis - MAPE-K).

(**RF4**) Decision Planning: It is necessary that the system has some way to deliberating on the events received, generating the necessary actions for the reestablishment of the system. (Decision Planning - MAPE-K).

(**RF5**) Execution: It is necessary that the system can act in the environment by changing the behavior of the managed element. (Execution - MAPE-K).

(**RF6**) Knowledge Base: It is desirable that the system has some kind of knowledge base to aid in decision making based on previously occurring facts. (Knowledge Base - MAPE-K).

(**RNF1**) Low coupling: This requirement is rooted in one of the basic principles of SOA.

B. *Intelligence Layer proposal*

According to nonfunctional requirement RNF1 (Low Coupling), the agents only act when a request is made, which typically occurs when the client notices a failure.

In the same way as the MAPE-K model there will be a Resource Monitored by the agents so that in case of failure, it can be recovered. Because it is possible to have several critical services, there may be several instances of agents to take care of each Monitored Resource. Therefore, each service can be a potential Monitored Resource.

The agents will also be built like services that will collaborate with each other to maintain the availability of other essential services that will be monitored. As the primary focus of this layer is to keep one or more essential SOA services always up and running, even when failures occur, the autonomous computing techniques that will be used will replace human interventions and automate any corrections.

The Agents will monitor their environment, so that when they receive any exception messages from a service, they first act to resolve the problem, such as restarting the failed service, and if this is not will be possible, its will act to find an alternative URL on UDDI so that the failed service can complete your request. For this can be occur, the agents must communicate with each other to deliberate on the best actions to be taken.

The Autonomic Agent Fig. 3 is responsible for passing the recovery requests to the instance of the Resource Monitor Agent corresponding to the Monitored Resource that it is being protected, so that it can take the necessary actions with the service that is failing, in addition, the Autonomic Agent should decide when to access the UDDI to switch to another service that can fulfill the request.

C. *Intelligence Layer Elements*

For each Monitored Resource (service) by the intelligence layer Figure 1 there must be an Autonomic Element according to the MAKE-K model. In this way, the Autonomic Element should perform the following tasks: Monitoring, Analysis, Plan, Execution, Query to Knowledge Base.

The proposed architecture divides the activities of the Autonomic Element into two agents that perform specific tasks: Autonomic Agent and the Resource Monitor Agent. All intelligence layers' elements are defined below:

- Autonomous Agent. This agent is responsible for the monitoring, analysis, planning and storage and query of knowledge.
- Resource Monitor Agent. This agent is responsible for the execution.
- Monitored Resource: This is the service or resource that will be monitored by agents. It is the Managed Element according to the MAKE-K model.

The division of tasks into two agents occurs due to the fact that the Monitored Resource is not always located in the same context (computer) of the caller. In this scenario, it is necessary to have a specific remote agent next to the service host machine. For this reason, the execution, in certain cases must be separated from the other actions of the agent.

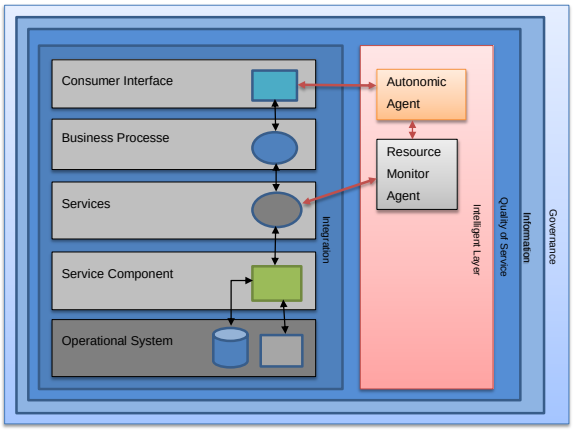

Fig. 3 Intelligence Layer Proposal (Produced by the author)

1) *Autonomic Agent*

In this proposal, the Autonomic Agent perception will not be through the continuous monitoring of its environment. The perception will occur through the receipt of exception messages. So, the Autonomic Agent is responsible for:

- (***M**onitor* $\mathbf{M}_{\text{APE-K}}$) messages from Resource Manager through exception calls from the service caller;
- (***A**nalyze* $_{M}\boldsymbol{A}_{PE\text{-}K}$) the messages to deliberate on the best actions and have been taken;
- (***P**lan* $_{MA}\boldsymbol{P}_{E\text{-}K}$) the actions that should be sent to the Resource Monitor Agent or to other actions, like call the UDDI when the default contingency actions fail;
- (***K**nowledge* $_{\text{MAPE-}}\mathbf{K}$). Store information resulting from its execution in its knowledge base to guide future actions.

The Autonomic Agent will also have the ability for switching client calls to another service as described in Pinto, Martins, Barbosa and Gluz [8]. This switching is critical when the contingency action (service restoration) fails. Information regarding decision-making from past deliberations will be stored in the knowledge base. This information will be used to know when it is necessary to try to recover a Monitored Element or when it is necessary to make a call to UDDI to find a new service that meets the request.

Thus, this agent is responsible for meeting the requirements (**RF3**) Data Analyze, (**RF4**) Decision Planning (**RF6**) Knowledge Base.

2) *Resource Monitor Agent*

The Resource Monitor Agent's sensor receives the messages coming from the Autonomic Agent and its actuator makes calls to the host of the Monitored Resource.

In addition to the message sensor, this agent has another sensor that monitors the service host environment to realize when the Monitored Resource is able to become responsive again after a failure event. So, the Resource Monitor Agent is responsible for:

- (***Executing*** MAP**E**-K) the necessary actions in the service's host, correcting the state of the Monitored Resource by restarting a stopped service.

This agent can be placed in the same context as the Autonomic Agent or can be placed remotely next to the Monitored Resource. The Resource Monitor Agent needs to have permission rights to act on the host. Because that, this Agent is always placed in the same context as the Monitored Resource.

This agent is responsible for meet the requirement (**RF5**) Execution.

3) Monitored Resource

The Monitored Resource can be any service that you want to monitor. Usually a SOA service has a host. This host can be: a server page, a window application, a console application, or an operating system's service.

Therefore, the Monitored Resource will be its managed by its host and the Resource Monitor Agent must have access to act in this host in order to make changes such as stop, start or restart the service

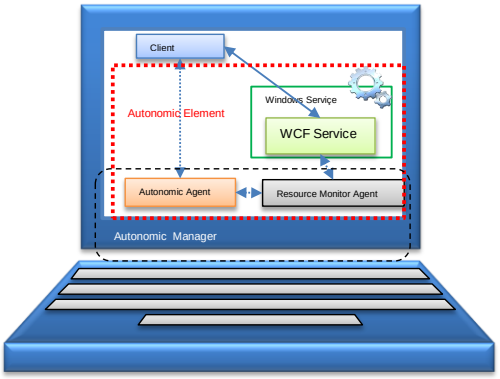

Fig. 4 Autonomic Architecture with local Monitored Element (Produced by the author)

D. *Autonomic Element vs MAPE-K Model*

Making a parallel between the proposed architecture and the MAPE-K model, we can see, from Fig. 4 that the Autonomic Element of the MAPE-K model is composed of all elements that is the Monitored Resource (WCF Service), the Autonomic Agent and the Resource Monitor Agent.

The Autonomic Manager of the MAPE-K model is composed of Autonomic Agent, the Resource Monitor Agent. The Managed Element of the MAPE-K model is composed of the Monitored Resource (WCF Service).

In a situation where Monitored Resource is hosted in another context (different computer), agents are distributed differently as shown in Fig. 5. Because of this, the Autonomic Element becomes fragmented.

Since the MAPE-K concept allow several Autonomic Elements, there could optionally exist an instance of the same Monitored Service on the computer where the Autonomic Agent is located that could be used for eventual contingency.

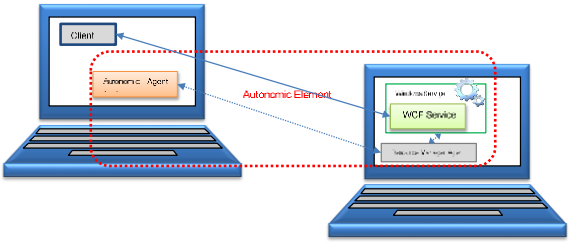

Fig. 5 Autonomic Element Architecture with Monitored Element (Produced by the author)

IV. Prototype Construction

The purpose of the developed prototype is to demonstrate and validate partially the architecture of the Intelligence Layer. This prototype consists of creating a real minimal SOA where agents act to solve possible failures in order to keep the business process running.

The prototype consists of an Orchestrator who calls three WCF services. The first and third calls are for a service that simply returns a random color. These calls serve to visualize the progress of the process as a whole. The second call is for a service that gets the computer name and the IP addresses where the service is running.

.

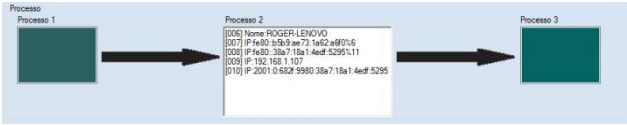

Fig. 6 Prototype Process Description (Produced by the author)

In this prototype, the service that returns the IP address is the Monitored Resource that will be protected by the Intelligence Layer agents. The complete process is quite simple and is presented in Fig. 6

The main flow of the prototype process consists basically of three calls made by an orchestrator. A call to a Monitored Resource (Process 2), which consists of a WCF that returns the computer name and its IP addresses, and two other auxiliary calls to an unmonitored WCF service that will only return a color in the RGB (Processes 1 and 3). The WCF service that returns the color is hosted on a web server, and its internal messages are not monitored. The Monitored Resource that returns the computer name and IP addresses will be hosted on an operating system service and will have its messages monitored.

A. *Prototype Architecture Description*

The prototype architecture is composed of an Orchestrator who manages the SOA process, an Autonomic Agent, a Resource Monitor Agent, a Monitored Resource that returns the computer name and IP address, and a Color Service. The following are the elements described above that make up the prototype architecture:

Orchestrator: It is the process that calls the other services, both the Color Service and the Monitored Resource. The Orchestrator's call sequence is shown in Fig. 6;

Monitored Resource: This WCF service returns the computer name and IP addresses. This service is hosted on the operating system services manager and is called by the Orchestrator;

Resource Monitor Agent: This agent manages and monitors the Monitored Resource host. It is responsible for restarting the service if it is necessary and for communicating the Autonomic Agent when the Monitored Resource becomes responsive after being inoperant for some time.

Autonomic Agent: This agent is responsible for exchanging messages with UDDI and sending the URL of the Monitored Resource to the Orchestrator. It receives the exception messages from the Orchestrator and is responsible for requesting the Resource Monitor Agent in order to correct them. The Autonomic Agent decides the best action to take according to the messages received by the Orchestrator and its internal states;

Color Service: This is an auxiliary service for better visualization of the process.

On the first computer, is running the Monitored Resource (WCF Service) that are hosted on Windows service, the color service that are hosted on IIS, the Orchestrator, the Autonomic Agent, and the Resource Monitor Agent. Otherwise, this computer is a host of a virtual machine with a Linux operating system that is running an instance of the Resource Monitor Agent and Monitored Resource (WCF Service). This computer will be referred to as the local computer.

B. Prototype Scenarios Description

The physical architecture of the prototype is composed of two computers that are interconnected by a wireless network **Erro! Fonte de referência não encontrada.**. The first computer named (Roger-HP) and the second computer named (Roger-Lenovo).

The second computer named (Roger-Lenovo) contains the Resource Monitor Agent and the Monitored Resource (WCF Service) that is hosted on the Windows Service. This computer will be referred to as the remote computer.

C. Test Scnarios

1) Remote Service Stopped

In this scenario, we observe the agents' performance. The remote server will be stopped and the agents will act towards the reestablishment of the system.

As can be seen in **Erro! Fonte de referência não encontrada.**, the process flow pass thought the Autonomous Agent and the Remote Resource Monitor Agent. In this case, unlike the previous scenario, if there is a failure, the process flow does not stop.

The Orchestrator's exception triggers the Autonomous Agent that makes a request to the remote Resource Monitor Agent to restart the remote Monitored Resource. After the restart, the Resource Monitor Agent returns the result of the operation to the Autonomous Agent.

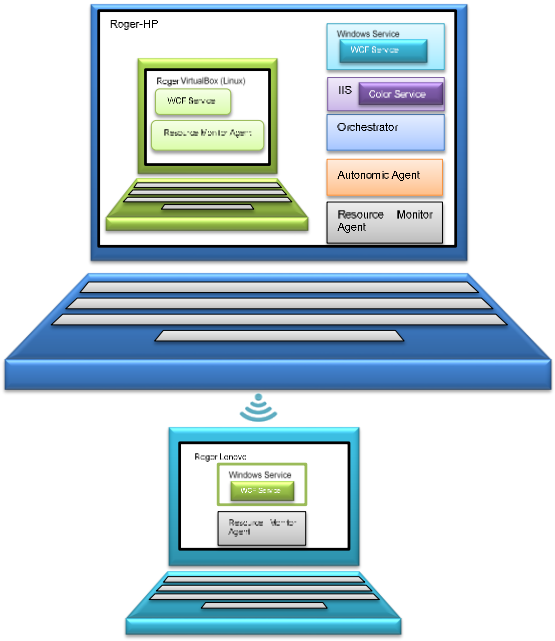

Fig. 7 Prototype Physical Architecture (Produced by the author)

The Autonomous Agent also returns the result to the Orchestrator who tries again to resume the process flow by calling the service that retrieve the computer name and IP addresses. If it succeeds the process flow continues and the color service is called again.

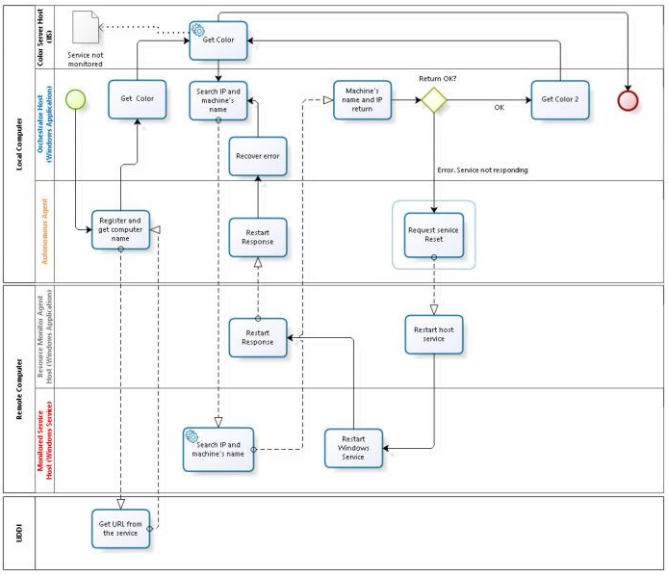

Fig. 8 Process using the agents and the remote service stopped (Produced by the author)

The process flow terminates successfully and with failure recovery by restarting the remote service host by the agents.

This scenario, as well as all others, is used to validate the following requirements:

- (**RF1**). Failure Detection

(**RF5**) Execution.

2) Remote Service Deactivated

In this scenario is simulated a situation in which the remote service is stopped and disabled. In this configuration, the restart of the service by the Resource Monitor Agent will fail because it will not be possible to reactivate the disabled service.

When the Autonomous Agent receives a restart request from the Orchestrator and the Resource Monitor Agent responds by failing to restart the service host, the Autonomous Agent will decide to find a new alternative to continue the Orchestrator's process. The solution in this case is to ask to the UDDI server an alternative service's URL that meets the search requirements of the computer name and IP addresses and that has the same interface as the previous service.

In **Erro! Fonte de referência não encontrada.** we can see a new condition next to the Autonomous Agent that checks if restart failed and if Resource Monitor Agent availability is available. In the event that this server is no longer attainable or a failure occurs, the Autonomous Agent decides to search for a new server through UDDI.

Upon receiving the new URL, the Autonomous Agent passes on this information to the Orchestrator who tries again to obtain the server name and IP addresses, but this time next to the alternative service. When getting into the computer name successfully, the color service can again be called.

In this case there should be an instance of the Monitored Resource on the computer that the UDDI should return.

The process terminates successfully without remote service recovery, but with redirection to an alternate server that is active and obtained through UDDI.

This scenario is used to validate the following requirements:

- (**RF3**) Data Analyze - When the Autonomous Agent parses the messages deciding when to restart or access UDDI.
- (**RF4**) Decision Planning - When the Autonomous Agent plans which messages to send for which elements (Resource Monitor Agent or UDDI)

3) Remote Service deactivated and local service stopped

In this scenario, a situation is simulated in which the Remotely Monitored Resource is stopped, the Monitored Resource host is disabled, and the local Alternative Monitored Resource obtained through the UDDI is also stopped.

In this situation, it is not possible to request a remote host restart of the Remotely Monitored Resource, because it is disabled and cannot access the local alternate Monitoring Service (WCF) service because it is stopped.

As can be seen from Figure 12, in the first interaction of the process flow, the monitored service (Search of Name and IP Address) fails. Because this condition, the flow checks whether the Resource Monitor Agent is available or if there was no error in the host restart process. In this case, the Remote Resource Monitor Agent is available but also fails because the host is disabled which leads the flow to obtaining a new alternate address in the UDDI by the Autonomous Agent.

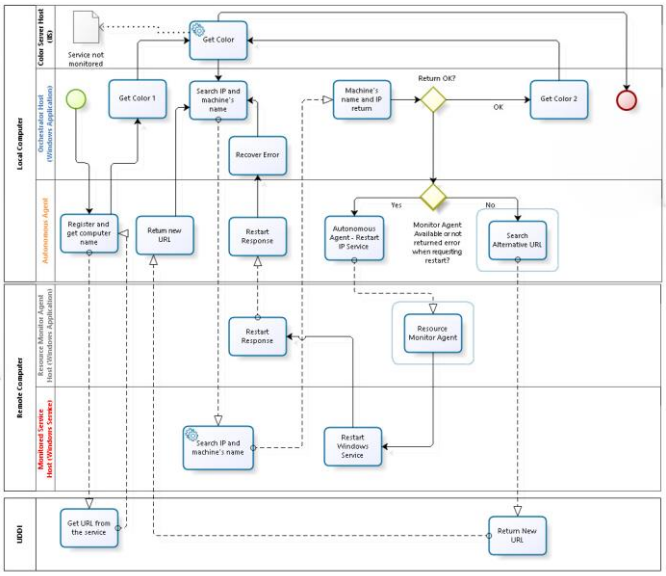

Fig. 9 Process with remote computer stopped and alternate computer active (Produced by the author)

In the second interaction of the process flow, now using the local alternate address, a fault again occurs, only now that the service is stopped. However, the Local Alternative Machine Resource Monitor Agent is active, which causes the process flow to attempt to restart Local Alternate Monitored Resource. This time it is possible to carry out the restart and the process flow follows with obtaining the name and the IP address only this time next to the local computer. After this process completes successfully, the process of obtaining the color can again be called to the process to finish successfully.

In the second interaction of the process flow, now using the local alternate address, a fault occurs again, now because that the local service is stopped. However, the local Alternative Machine Resource Monitor Agent is active, which causes the process flow to attempt to restart Local Alternate Monitored Resource. This time it is possible to carry out the restart and the process flow follows with obtaining the name and the IP address only this time next to the local computer. After this process completes successfully, the process of obtaining the color can again be called to the process to finish successfully.

The process terminates successfully without recovery of the remote service, but with the recovery of the local alternative service that was obtained through the UDDI that was stopped.

This scenario is used to validate the requirement (**RF6**) - Knowledge Base, because based on information stored in the Knowledge Base, the UDDI could be triggered.

4) Remote Service deactivated and local service deactivated

In this scenario, we will show how the architecture behaves when the Remote and Local Monitored Resources fail. In this case the remote computer cannot be reached and the local alternate service found by UDDI does not respond because it is stopped, and the local host cannot be restarted because it is disabled. However, in this scenario, there is still a new possibility of an alternate URL that is in the virtual machine with Linux operating system that will respond to the request.

As can be seen from **Erro! Fonte de referência não encontrada.**, in the first interaction of the process flow the Remote Monitored Resource (Search of Name and IP Address) fails. Because this condition, the flow checks whether the Resource Monitor Agent is available or if there was no error in the host restart process. In this case the Remote Resource Monitor Agent is active, but the remote service host is disabled, which leads to obtaining a new alternate address in UDDI.

In the second interaction of the process flow, now using the local alternate address, again a failure occurs because the local service is also disabled. This takes the process flow to the third interaction that searches for a new alternate URL. This URL is found in the virtual machine and the process flow follows with obtaining the name and the IP address. After this process completes successfully, the process of obtaining the color can again be called to the process to finish successfully.

The process terminates successfully without remote service recovery or local service recovery, but with redirection to an alternate server retrieved from the virtual machine through UDDI that is active.

5) Remote service deactivated, local service deactivated and Virtual Machine service stopped

In this scenario, we will show how architecture behaves when all Monitored Resources fail. Local and Remote Monitoring Resource hosts are disabled. The Virtual Machine Monitored Resource is stopped, but the Virtual Machine Resource Monitor Agent is active.

Before the process can start it is important to start the Virtual Machine Resource Monitor Agent.

Este cenário é o mesmo que antes, e quando o fluxo encontra o serviço de máquina virtual parado, ele aciona o agente do monitor de recursos que reinicia o serviço.

The process terminates successfully without remote service recovery and local service recovery, but with virtual machine service recovery by the Virtual Machine Resource Monitor Agent.

6) Remote service deactivated, local service deactivated, Virtual Machine service stopped and Resource Monitor Agent's Virtual machine off.

In this scenario, we will show how architecture behaves when all Monitored Resources fail. Remote and local monitored host is disabled. The Monitored Resources of the virtual machine is stopped and the Virtual Machine Resource Monitor Agent is off.

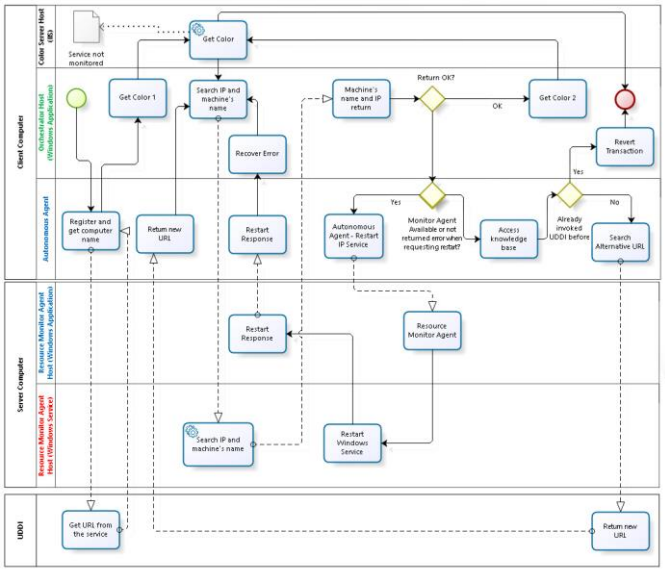

Fig. 10 Process with no answering service (Produced by the Author)

As we can see in Fig. 10 a new condition appears in order to be able to insert the concept of transaction by compensation. In the first interaction, the Remote Service Resource Monitor Agent does not respond so the Autonomous Agent searches for a new URL in the UDDI. This interaction can be repeated while UDDI can respond to new alternate URLs. Since the process flow passes through a URL that has been tried previously, the information that was obtained through the query in the knowledge base tell that there are no more new URLs to be retrieved. Therefore, the process flow cannot continue and the transaction must be undone. The Autonomous Agent in this case sends a message to the Orchestrator telling that a compensation transaction must be triggered.

The process terminates with error, but with a compensation transaction being triggered. This keeps the transaction state consistent, even though no Monitored Resource can be recovered.

This scenario serves to validate the requirement (**RF2**) Transaction.

D. Requirement Validation

The following Table 2 summarizes the intelligence layer requirements and how they can be achieved:

TABLE I. VALIDATION OF REQUIREMENTS (PRODUCED BY THE AUTHOR)

RF1	Failure Detection	Using exception handling	It is necessary to create a failure situation and verify that the intelligence layer is able to detect the failure.
RF2	Transaction	Using Transaction Mechanism	It is necessary to create a situation where it is not possible to recover the failure and check if the system has returned to a consistent state.
RF3	Data Analyze	It will be done by the **Autonomous Agent**	It is necessary to verify that the messages received by the **Autonomous Agent** are analyzed and generate the appropriate plans according to the

			execution context.
RF4	Decision Planning		Verify that the requests to the **Resource Monitor Agent** and to the UDDI that were planned by the **Autonomous Agent** are being sent correctly.
RF6	Knowledge Base	It will be done by the **Autonomous Agent**	Verify that the requests to the **Resource Monitor Agent** and to the UDDI that were planned by the **Autonomous Agent** are being sent correctly.
RF5	Execution	It will be done by the **Resource Monitor Agent**	Verify that the requests to the **Resource Monitor Agent** and to the UDDI that were planned by the **Autonomous Agent** are being sent correctly.
RNF1	Low coupling	Through interfaces	Show that is possible to use the intelligence layer optionally.

V. Results Analysis

The main objective of the prototype was to present some practical cases of use of the intelligence layer and also. The prototype allows visualizing the agents during the processes of self-healing and self-management.

A. Main Prototype Results

It can be observed through the scenarios presented in section IV, Prototype Construction that the autonomous computation's techniques allied to multiagent techniques, increased the stability, the quality and the availability of the system.

We observed that when the Agents decided to search for alternative URLs through the UDDI search, they presented a self-management behavior and when they reset the hosts of the services that were stopped, they presented a self-healing behavior.

In this way, the prototype was extremely useful for visualizing the detailed functioning of the intelligence layer and the exchange of messages between agents.

This paper we only proposes a simple form of deliberation for failure recovery, however, it is possible to combine several other artificial intelligence techniques to create more sophisticated deliberative processes.

VI. Conclusion

This work presented a proposal for an extension of the service quality layer of the SOA model proposed by [2] for increased availability. This extension was called intelligence-layer and uses autonomous computing techniques based on the MAPE-K model and multiagent techniques.

The Intelligence Layer proposed is capable for performing self-healing and self-management of the services of an SOA and also proved to be quite adequate for the purpose presented when recovering the services in several failure cases.

It was possible to observe that SOA offered a good infrastructure for building autonomous computing based on agents because of its messaging infrastructure and for being self-contained.

It can also be observed that autonomous computing can be constructed using agents to increase availability, reliability and self-management of business processes.

The intelligence layer created a parallel processing to solve the problems of unavailability when failures in the hosts of the services occurred. This allowed the system to be restored automatically, and without user interference. When the agents acted towards the reestablishment of the system were possible conclude that agents combined with autonomous computing proved to be quite efficient for the construction of fault-tolerant architectures.

Franke [4] in his work conclude that short interruptions to wait the service restart, proved to be better than long interruptions when the interruptions' cost is proportional of interruptions duration. Thus, this work, even showing brief interruptions to restart the service, has proved to be effective in increasing the quality of service and availability of SOA

It is possible conclude that the proposed architecture can serve as a reference for the construction of SOA architectures that have services of high criticality, that have a strong dependence on other services and that require high reliability and high availability for their business processes. As the intelligence layer decreases the time between failures, since there is no longer a need for human action, service contract (SLA) can be established taking into account a shorter time between the corrections of failures.

References

[1] M. P. Papazoglou e D. Georgakopoulos, "Service-Oriented Computing:Concepts, Characteristics and Directions," em *Proceedings of the Fourth International Conference on Web Information Systems Engineering*, 2003.

[2] Open Group, "Using TOGAF to Define & Govern SOAs," 2011. [Online]. Available: http://pubs.opengroup.org/architecture/togaf9-doc/arch/chap22.html. [Acesso em 17 09 2016].

[3] M. P. Papazoglou, P. Traverso, S. Dustdar e F. Leymann, "Service-oriented computing: State of the art and research challenges," *IEEE*, vol. 40, pp. 64-81, 2007.

[4] U. Franke, "Optimal IT service availability: Shorter outages, or fewer?," *IEEE Transactions on Network and Service Management*, vol. 9, pp. 22-33, 2012.

[5] A. Computing, "An architectural blueprint for autonomic computing," *IBM White Paper*, vol. 31, 2006.

[6] J. O. Kephart e D. M. Chess, "The vision of autonomic computing," *IEEE*, vol. 36, pp. 41-50, 2003.

[7] M. C. Huebscher e J. A. McCann, "A survey of autonomic computing—degrees, models, and applications," *ACM Computing Surveys (CSUR)*, vol. 40, p. 7, 2008.

[8] S. C. C. S. Pinto, R. S. M. Martins, J. L. V. Barbosa e J. Gluz, "Composição dinâmica de Web-Services," *XXII Simpósio Brasileiro de Engenharia de Software*, pp. 220-235, 2008.